How do you ensure code quality and best practices in your Drupal projects?

Drupal projects requires a combination of coding standards, tooling, architectural patterns, review processes, and automation.

Below is a complete, real-world checklist used by experienced Drupal teams.

✅ 1. Follow Drupal Coding Standards

Drupal follows strict PHP coding standards:

▸ **Tools**

PHPCS with Drupal standards

```
composer require --dev drupal/coder

phpcs --standard=Drupal,DrupalPractice web/modules/custom
```

- Enforce coding style (spaces, naming, class structure)
- Ensures consistency across team members

✅ 2. Use Static Analysis Tools

Static analysis helps catch bugs early.

▸ **Recommended tools:**

PHPStan (with Drupal extension)

```
composer require --dev phpstan/phpstan mglaman/phpstan-drupal

vendor/bin/phpstan analyse web/modules/custom
```

- **PHP Mess Detector (PHPMD)**
- **ESLint** / **StyleLint** for JS and CSS

These tools catch:

- Undefined methods
- Wrong return types
- Schema or service issues
- Deprecated APIs

✅ 3. Enable and Use Devel / Webprofiler (During Development Only)

Helps identify:

- Slow code
- Inefficient database queries
- Broken cache metadata
- Missing cache contexts

This ensures code is performant and cache-friendly.

✅ 4. Use Proper Drupal API and Architecture

Follow Drupal's preferred patterns:

- **Dependency Injection**, NOT `\Drupal::service()` in classes
- Use **services**, **plugins**, **events**, **hooks** properly
- Avoid hardcoding configuration
- Use **config API**, NOT settings.php
- Write **cacheable metadata** (`#cache`, cache tags, contexts)

✅ 5. Write Tests (Automated)

Automated testing is essential:

▸ **Types of tests**

- **Unit tests** (PHPUnit)
- **Kernel tests** (Drupal bootstrapped)
- **Functional tests** (Mink, JavaScript testing)
- **Behat** for behavior testing

Run tests:

```
phpunit --testsuite unit
```

✅ 6. Use CI/CD pipelines

Common tools: • GitHub Actions • GitLab CI • Bitbucket Pipelines • CircleCI	CI runs automatically: • PHPCS • PHPStan • PHPUnit • Security scan • Linting

This ensures no low-quality code is merged.

✅ 7. Use Automated Deployment / Configuration Management

Using Drupal Config Management (CMI): drush cex drush cim	Best practices: ● Export config to version control ● Review config changes in merge requests ● Do NOT modify config manually on production

---

✅ 8. Use Hook Requirements and Uninstall hooks

Ensure modules clean up after themselves:

- `hook_requirements()`
- `hook_uninstall()`
- `hook_schema()`
- `hook_update_N()` for database changes

Keeps site stable and clean.

✅ 9. Documentation and Commenting

- Use PHPDoc and proper comments
- Document:
  - Custom services
  - Plugin availability
  - Important business logic
  - Configurable behavior

Good documentation increases maintainability.

✅ 10. Code Review (Peer Review)

Every change should go through:

- Merge request review
- Visual testing
- Performance testing
- Functional QA testing

✅ 11. Security Best Practices

- Use Drupal Security Advisories (PSA)

Run:

```
drush pm:security
```

-
- Avoid raw SQL if possible
- Sanitize output via Twig autoescape
- Validate user input
- Avoid exposing sensitive config

⚡ Summary: Best Practices Checklist

Category	Tools / Guidelines
Coding Standards	PHPCS, Drupal Coder
Static Analysis	PHPStan, PHPMD
Performance	Webprofiler, caching system
Architecture	DI, plugins, services
Testing	PHPUnit, Kernel tests, Behat
CI/CD	Automated scans + tests
Config	CMI (drush cex/cim)
Documentation	PHPDoc, comments
Security	PSA, drush security

AWS Service Knowledge Overview

Amazon S3 (Simple Storage Service)

- Object storage for any type of data (images, logs, backups, static sites, etc.)
- Highly durable (11 nines), scalable, and cost-effective.
- Supports versioning, lifecycle rules, encryption (SSE-S3, SSE-KMS), and bucket policies.
- Often used for static website hosting, big data storage, and integration with many AWS services.

Amazon EC2 (Elastic Compute Cloud)

- Resizable virtual machines in the cloud.
- Provides wide instance families (General Purpose, Compute Optimized, Memory Optimized, GPU, etc.).
- Features include Auto Scaling Groups, Elastic Load Balancing, EBS volumes, and user-data for bootstrapping.
- Used for hosting applications, servers, microservices, and more.

AWS Lambda

- Serverless compute service—runs code without provisioning servers.
- Supports multiple languages (Node.js, Python, Java, Go, etc.).
- Automatically scales with demand; pay only for execution time.
- Commonly used for event-driven architectures, API backends, data processing, and automation.

Amazon RDS (Relational Database Service)

- Managed relational databases (MySQL, PostgreSQL, Oracle, SQL Server, MariaDB, and Aurora).
- Automates backups, patching, failover, and scaling.
- Multi-AZ for high availability; read replicas for scaling reads.
- Reduces administrative overhead for database management.

Architecture diagrams for Headless Drupal 11

Below are **three clear, production-grade architecture diagrams** for **Headless Drupal 11** implementations.
They include:

- API layer
- CDN caching
- Front-end frameworks (React, Next.js, Vue, mobile apps)
- Authentication flows
- Optional microservices
- Workflow and scalability patterns

All diagrams use **ASCII architecture maps** so they can be easily copied into documentation.

🧱 1. Standard Headless Drupal 11 Architecture

Use case: Traditional decoupled site with React / Vue / Svelte front-end.

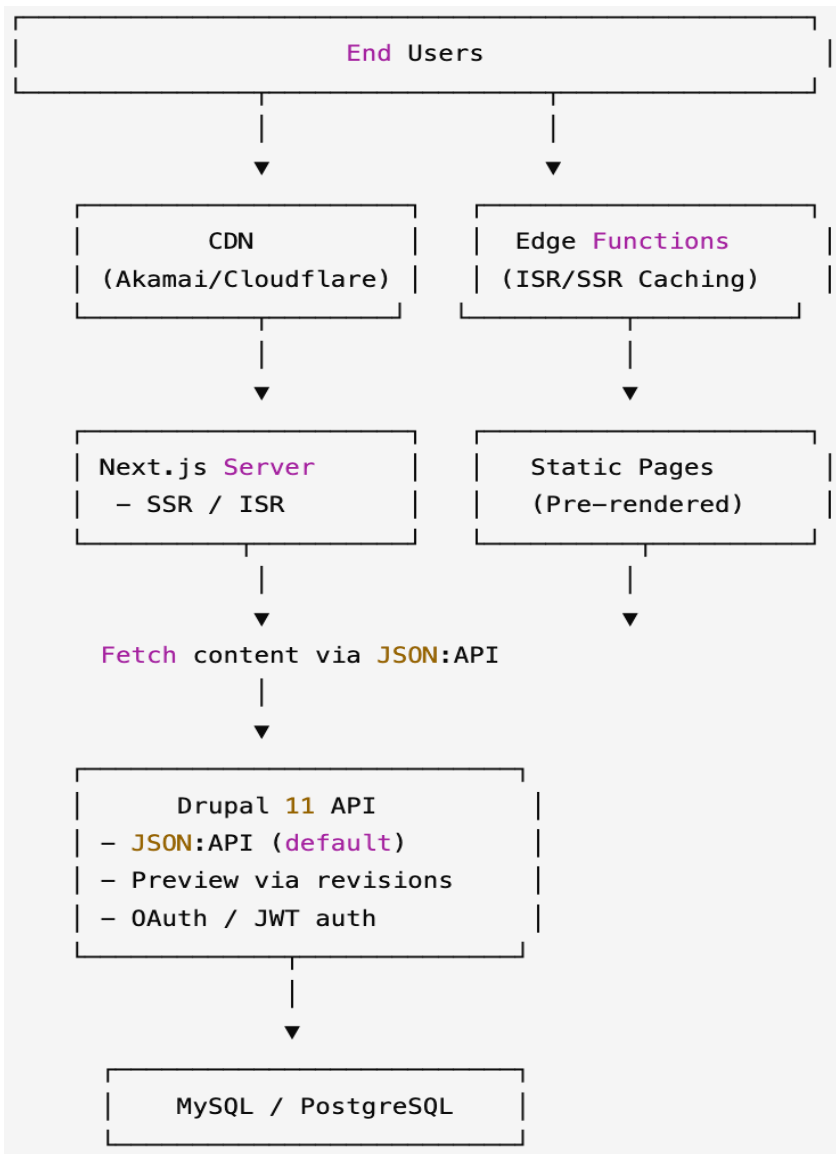
```
┌──────────────────────────┐
│     Front-End App        │
│ (React / Vue / Svelte)   │
└──────────────────────────┘
              │
              ▼
     JSON:API / REST
              │
              ▼
┌──────────────────────────┐
│       Drupal 11          │
│    (Content Backend)     │
│  - JSON:API (default)    │
│  - REST Resources        │
│  - Custom Controllers    │
└──────────────────────────┘
              │
              ▼
      Drupal Services
     (Entities, Cache,
      Permissions, etc.)
              │
              ▼
┌──────────────────────────┐
│   Database (MySQL/Maria) │
└──────────────────────────┘
```

Key points

- Front-end pulls data from `/jsonapi/...`
- No server-side rendering; pure API consumption
- Best for SPAs, hybrid mobile apps

⚡ 2. Headless Drupal 11 with Next.js / Nuxt SSR + CDN

Use case: High performance, SEO-friendly, scalable headless architecture.

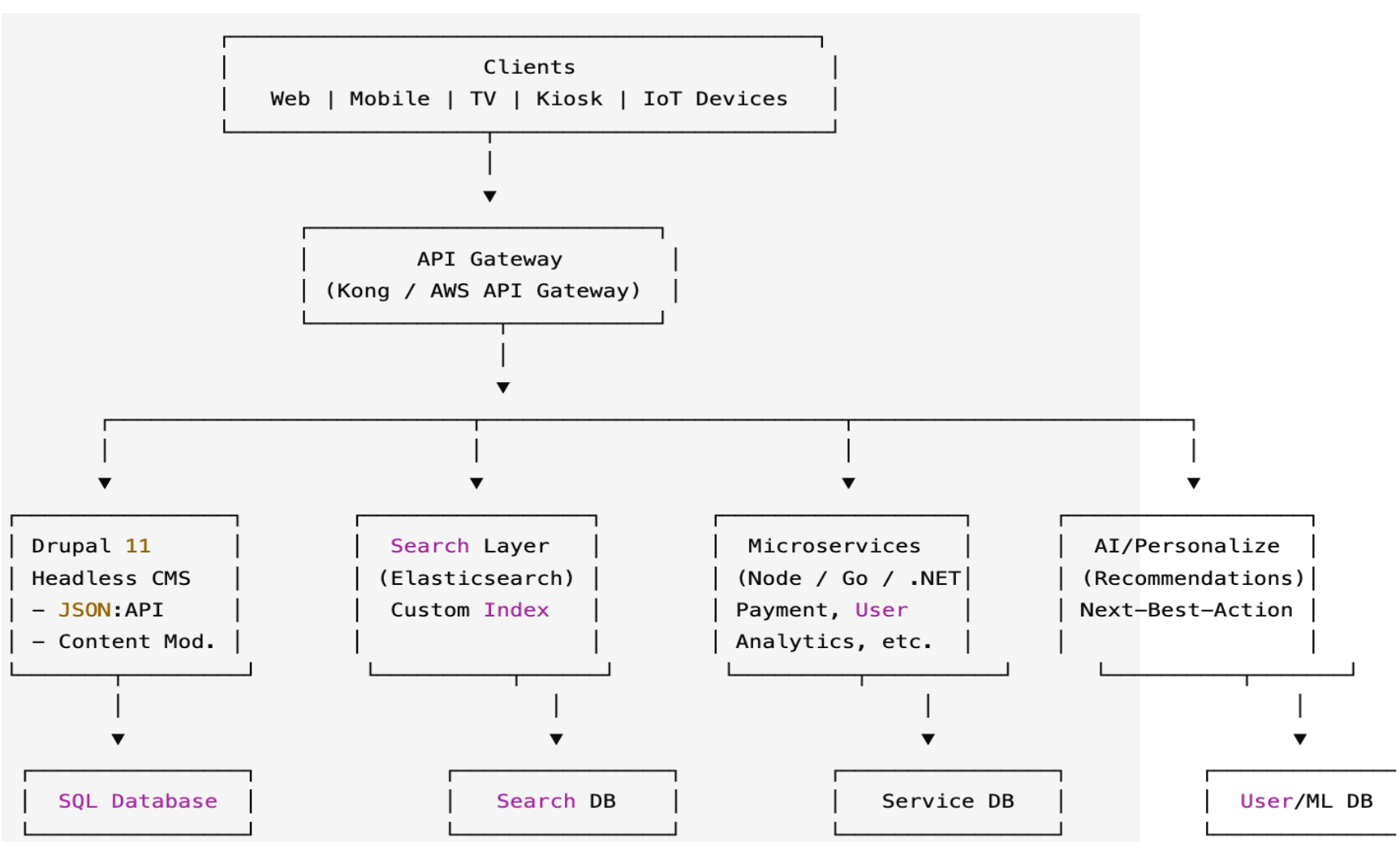


Key points

- **Next.js handles SSR/ISR → SEO optimized**
- Data fetched from JSON:API at build time or runtime
- **CDN caching drastically reduces Drupal load**
- Supports **preview mode** via Drupal revision routes

🔥 3. Enterprise Microservices + Drupal 11 as Content Engine

Use case: Enterprise-grade architecture with multiple consumers (mobile, kiosks, websites).



Key points

- **API Gateway** manages versioning, rate limits, JWT, throttling
- Drupal supplies structured content only
- Other services handle:
  - search
  - payments
  - personalization
  - commerce logic
- Ideal for massive enterprise systems

🎯 Optional Diagram: Content Preview Architecture

(Editors want Next.js preview for drafts)

```
        Drupal Editor
              |
     Click "Preview"
              ▼
┌──────────────────────┐
| Drupal 11 Backend    |
| - Revisions          |
| - JSON:API preview   |
└──────────────────────┘
              | Signed preview token
              ▼
        Preview URL
              ▼
┌──────────────────────┐
| Next.js Preview      |
|  - Disables cache    |
|  - Fetches draft     |
└──────────────────────┘
              |
              ▼
          JSON:API
```

This flow enables:

- draft-only visibility
- secure preview tokens
- real-time editorial preview

Why is dependency injection recommended?

Makes code testable

Avoids global Drupal::service()

Follows SOLID principles

Better performance via lazy loading explain in details